ButterPop!

Unity에 없는 소프트바디를 직접 구현해 스토어 출시까지

쫀득한 스퀴시의 손맛과 ASMR을 느낄 수 있는 모바일 2D 물리 퍼즐

플레이스토어 플레이영상 Github

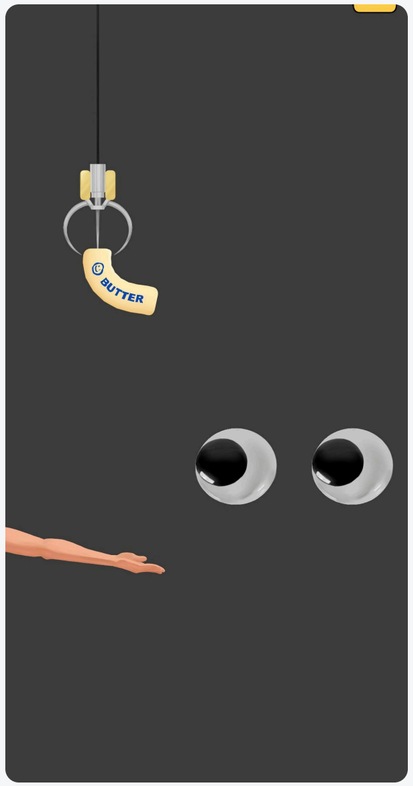
개발 기간
26.06 ~ 26.09 (3개월)

팀 구성
4인 (기획 · 아트 · 마케팅 · 개발)

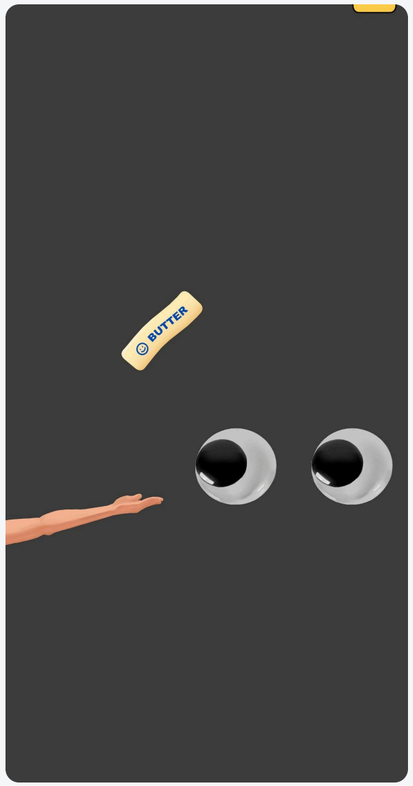
담당 역할
메인 클라이언트 개발

개발 환경
Unity 6 · URP · C# / Android

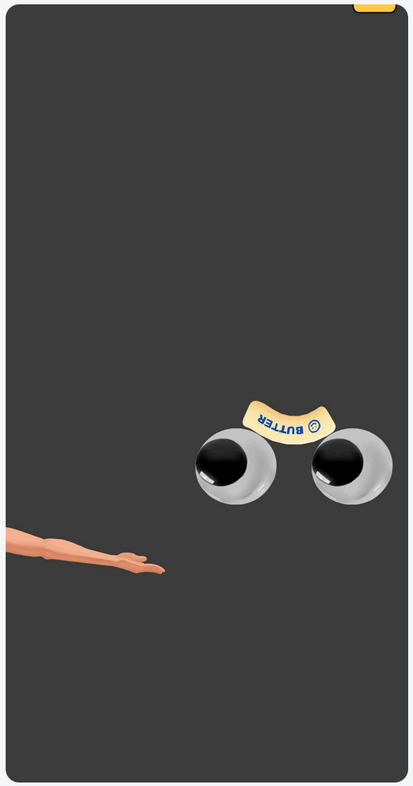
플레이 화면



스퀴시 떨어뜨리기



손으로 쳐서 올리기



스테이지 위에 안착

핵심 구현

① 쫀득한 2D 소프트바디

휘어짐은 HingeJoint2D 체인, 출렁임은 런타임 메시로 구현

② 스테이지 제작 파이프라인

기획자가 코드 없이 스테이지를 제작·수정할 수 있도록 파이프라인 구축

③ 인터페이스 기반 코어 아키텍처

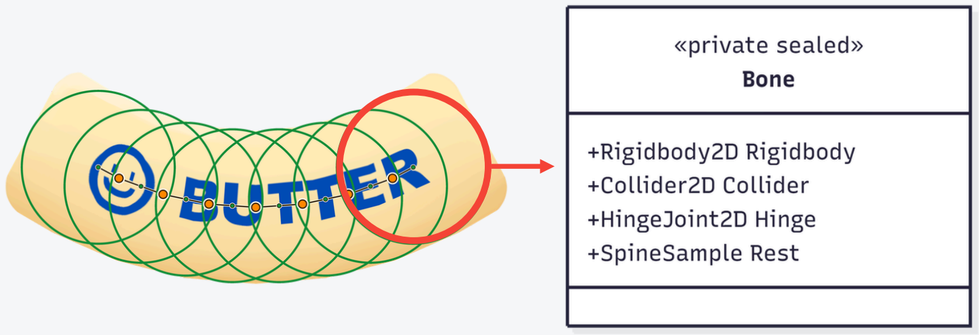
게임 로직은 Service, SDK는 Provider로 분리

④ 출시 서비스 문제 대응

스토어 무응답·저장 충돌·광고 로드 실패를 서비스별로 차단

핵심 구현 ① 쫀득한 2D 소프트바디

Unity 2D에 없는 스퀴시 소프트바디 물리 직접 구현



Collider를 가진 Bone을 HingeJoint2D로 연결한 물리 체인

```
private void ApplyBendingSprings()
{
    for (int i = 0; i < _bones.Count - 1; i++)
    {
        var a = _bones[i].Rigidbody;
        var b = _bones[i + 1].Rigidbody;

        float errDeg = Mathf.DeltaAngle(a.rotation, b.rotation);
        float relVelDeg = b.angularVelocity - a.angularVelocity;

        float t = -bendStiffness * (errDeg * Mathf.Deg2Rad)
            - bendDamping * (relVelDeg * Mathf.Deg2Rad);

        b.AddTorque(t * b.inertia);
        a.AddTorque(-t * a.inertia);
    }
}
```

스프링(복원) + 댐퍼(출렁임 억제)

ApplyBendingSprings() : 이웃 Bone마다 복원 토크 적용

설계 목표 - 스퀴시처럼 탄력 있는 소프트바디 물리

- 외부 압력에 휘어지고 탄력 있게 복원되는 몸체
- 강하게 부딪혀도 형태가 무너지거나 관통하지 않는 안정성
- 코드 수정 없이 수치만으로 탄력감을 조정할 수 있는 구조

구현 내용 - Bone 관절 체인 + 복원 토크

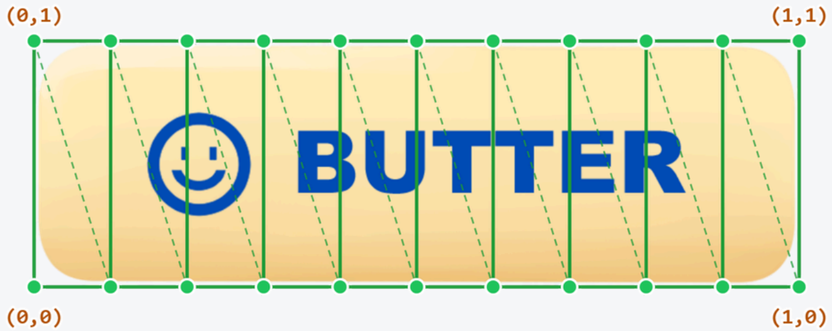
- 여러 Bone을 HingeJoint2D로 연결해 마디마다 휘어지는 구조
- HingeJoint2D엔 복원력이 없어 복원 토크를 직접 구현
- 스프링 토크로 일자 복원, 댐퍼로 출렁임 억제
- 관절 각도 제한과 Continuous 충돌 감지로 과도한 관절 변형과 관통 가능성 감소

검증 및 효과

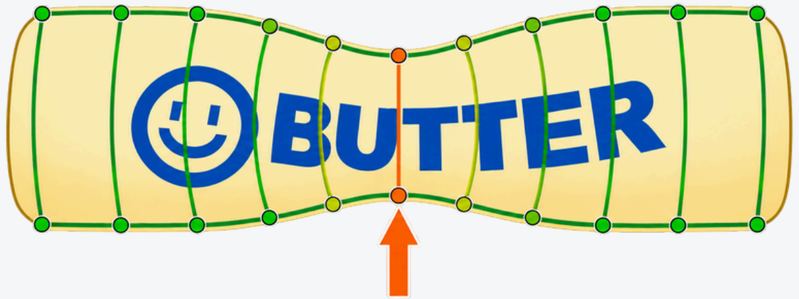
- **휘어졌다 탄력 있게 복원되는 스퀴시 몸체를 Unity 2D에서 구현**
- **스프링·댐퍼 값 조정만으로 탄력감 튜닝 가능**

핵심 구현 ① 쫀득한 2D 소프트바디

런타임 메시 변형으로 구현한 말랑한 스퀴시 외형



정점 배치 + UV 매핑으로 만든 메시



충돌점 주변 단면의 단계적 압축과 메시 갱신

설계 목표 - 물리와 충돌에 반응해 변형되는 외형

- 스프라이트로는 표현할 수 없는, 매 프레임 휘고 눌리는 몸체
- 안착 판정이 흔들리지 않도록 물리와 렌더링을 분리한 구조

구현 내용 - Sprite Texture를 입힌 메시를 정점 이동으로 변형

- Bone 체인을 따라 정점을 배치하고, UV로 Texture를 매핑해 메시 생성
- UV는 고정한 채 매 프레임 정점만 옮겨, 충돌에 따른 모습을 그대로 렌더링
- 충돌점 중심으로 멀어질수록 눌림이 약해지고, 감쇠 스프링으로 복원
- 물리를 따라 정점만 수정해 그릴 뿐, Collider는 건드리지 않음

검증 및 효과

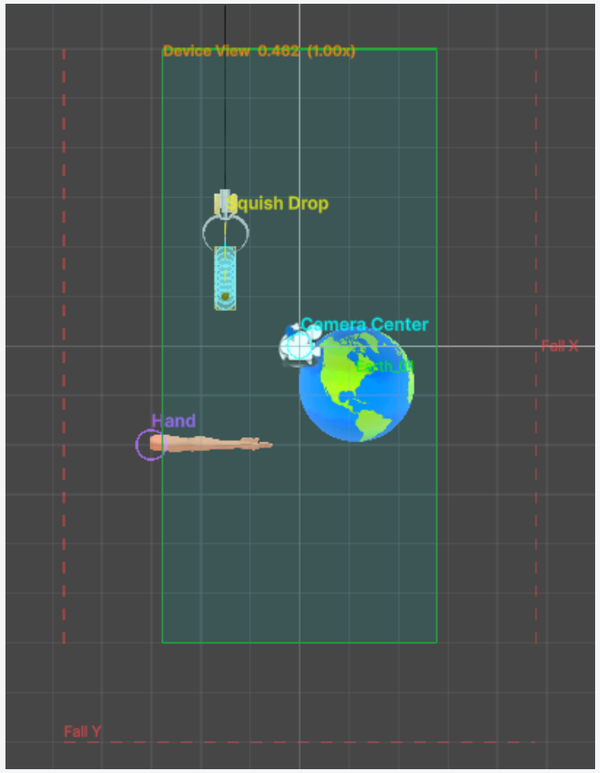
- **Sprite 한 장을 입힌 메시로 휘고 눌리는 모습을 매 프레임 표현**
- **외형 변형이 Collider와 안착 판정에 영향을 주지 않도록 분리**

핵심 구현 ② 스테이지 제작 파이프라인

스테이지 제작 툴로 50개 스테이지 구축



스테이지 메이커 툴



씬뷰를 이용한 오브젝트 배치

설계 목표 - 기획자가 직접 제작 및 수정할 수 있는 환경 구축

- 기획자가 코드 수정 없이 스테이지를 직접 제작 및 수정
- 씬뷰에서 결과를 확인하며 오브젝트 배치부터 데이터 저장까지 하나의 흐름으로 처리

구현 내용 - 실시간 프리뷰와 데이터 기반 스테이지 구성

- 오브젝트의 위치·회전·크기 등을 조정하고 씬뷰에서 실시간 확인
- 화면비율에 따라 실제로 표시되는 영역을 씬뷰에서 실시간 확인
- 스프라이트 등록 시 Collider를 포함한 StageObject 프리팹 자동 생성
- 제작한 스테이지를 StageDataSO로 저장하고 StageBuilder가 스테이지를 재구성

검증 및 효과

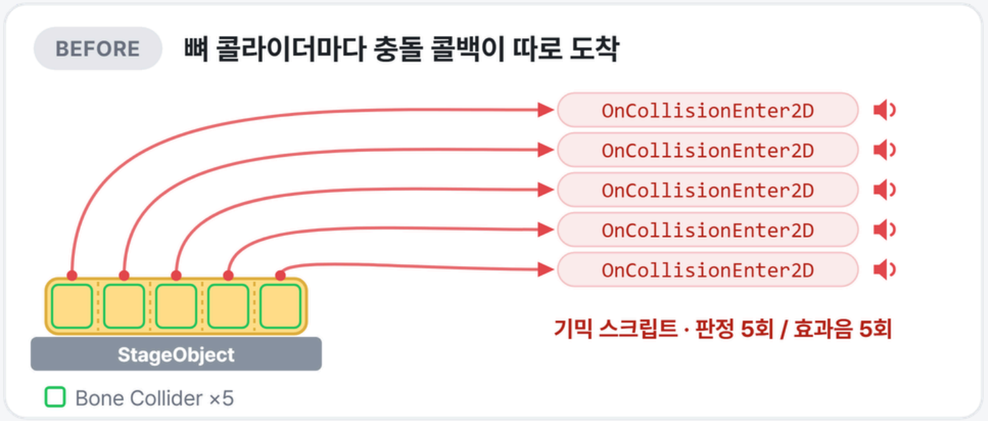
- **동일한 제작 흐름으로 50개 스테이지를 제작하고 실제 플레이 환경에서 검증 완료**

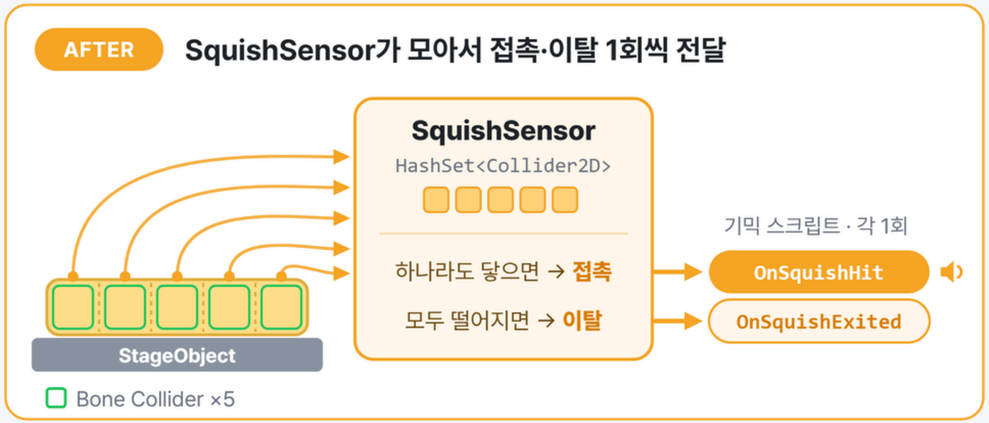
핵심 구현 ② 스테이지 제작 파이프라인

스테이지 오브젝트의 충돌 반응을 공통 컴포넌트로 구성

문제 발생

- 스퀴시가 스테이지 오브젝트에 한 번 닿았는데 효과음이 여러 번 재생되고 기믹 반응이 중복 실행됨





원인 분석

- 스퀴시는 뼈마다 Collider가 있어 한 번 닿아도 콜백이 여러 번 발생
- 기믹마다 충돌 감지를 따로 구현해 중복 코드 발생

해결 방법

- SquishSensor가 닿은 Bone Collider를 HashSet으로 추적
- 첫 Bone 접촉 시 Enter, 마지막 Bone 이탈 시 Exit 이벤트를 각각 한 번만 전달
- 물리 충돌은 ContactSensor, 통과 영역은 TriggerSensor로 분리

적용 효과

- **Bone 개수와 관계없이 접촉·이탈 이벤트가 한 번씩만 발생**
- **효과음 중복 재생과 기믹 반응 중복 실행 해소**

핵심 구현 ③ 인터페이스 기반 코어 아키텍처

플랫폼·SDK가 바뀌어도 게임 코드는 그대로



설계 목표 - 플랫폼·SDK에 독립적인 코어 구조

- 광고·결제 SDK는 플랫폼별로 다르고 에디터에선 동작하지 않음
- 플랫폼·SDK가 바뀌어도 게임 코드는 수정하지 않는 구조 목표

설계 선택 - 인터페이스 + ServiceLocator

- 서비스는 인터페이스로 정의, 구현체는 Bootstrap에서 선택
- 빌드 조건에 따라 실제 SDK / Stub 구현체를 교체
- 서비스 규모에 맞춰 외부 의존 없는 경량 로케이터 채택
- 인터페이스 기반 등록으로 호출부 수정 없이 구현체 교체

검증 및 효과

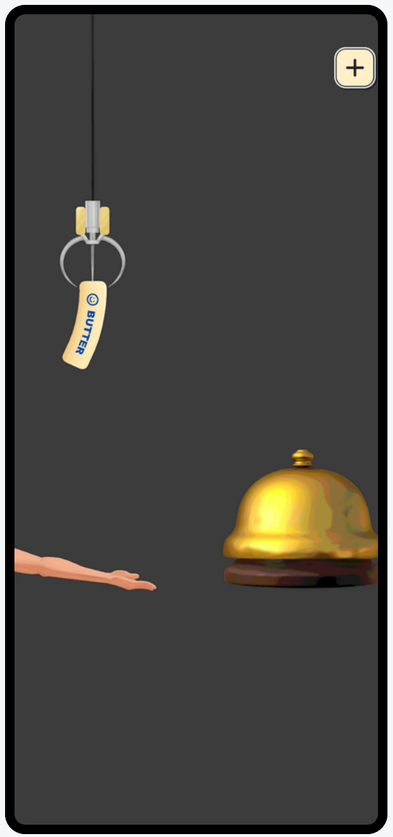
- **플랫폼별 구현체와 서비스 계층을 확장해 SDK 연동, 게임·UI 코드는 유지**
- **플랫폼·빌드 조건에 따라 서비스별 실제 SDK·Stub 구현체 선택**

핵심 구현 ④ 출시 서비스 문제 대응

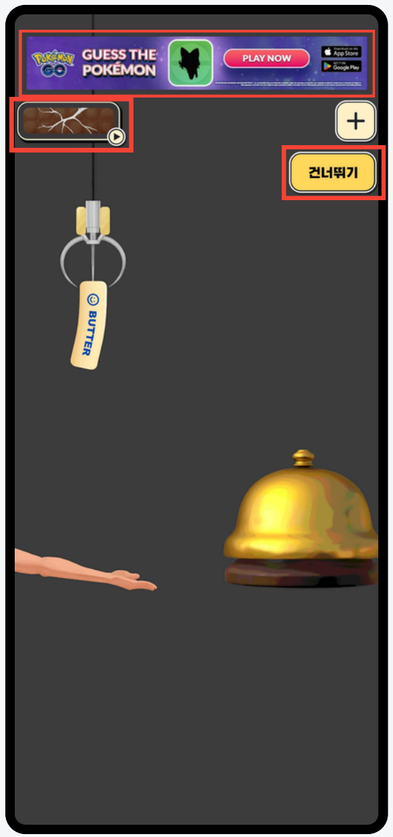
광고 로드 상태를 추적해 배너 재요청·리워드 노출 제어

문제 발생

- 배너는 최초 로드가 실패하면 세션 내내 표시되지 않고, 리워드는 준비되기 전에도 버튼이 노출돼 눌러도 반응이 없음



광고 미준비 — 비행기 모드



광고 준비 완료

원인 분석

- 배너를 초기화 때 한 번만 로드
- 배너 객체 존재만 보고 실제 노출 여부를 판단
- 리워드 준비 여부와 상관없이 버튼을 노출

해결 방법

- 배너 광고 최초 로드 실패 시 5·15·30·60초로 간격을 늘려가며 재요청
- 배너 객체 대신 요청·로드·노출 상태를 bool로 직접 관리
- 리워드 광고 준비 상태 변경 시 이벤트로 버튼 노출 제어

적용 효과

- **첫 로드 실패 후에도 재요청을 지속해 세션 중 복구 가능**
- **리워드 광고가 준비되지 않으면 버튼을 노출하지 않음**

핵심 구현 ④ 출시 서비스 문제 대응

미확정 주문 복원과 구매 중 무한 대기 차단

01 미확정 주문 복원

결제 승인 직후 앱이 종료되면 주문이 미확정 상태로 남음

원인 분석

- 구매 확정 전 앱 종료로 처리 흐름이 중단됨

해결 방법

- 세션 시작 시 보유 주문을 조회해 미확정 주문 확정 및 권한 복구

적용 효과

- **결제 직후 앱이 종료돼도 다음 실행에서 주문 처리를 완료하고 구매 권한 복구**

02 구매 중 무한 대기 차단

스토어 무응답·처리 예외로 구매 대기가 지속되고 후속 구매 차단

원인 분석

- 콜백 기반 API를 await로 감싸면 무응답 시 대기가 풀리지 않음
- 대기 핸들이 해제되지 않으면 이후 구매가 전부 차단

해결 방법

- 연결·조회 30초, 결제 900초 타임아웃으로 무한 대기 차단
- 예외·타임아웃 시 대기 핸들을 해제해 후속 구매 차단 방지

적용 효과

- **예외·무응답 발생 시에도 대기 상태를 해제해 상점 잠김 방지**
- **응답이 없어도 구매 대기를 종료하고 재시도 가능한 상태로 복귀**

핵심 구현 ④ 출시 서비스 문제 대응

저장 파일 손상과 동시 저장 충돌 방지

01 저장 중단 시 파일 손상 방지

저장 중 앱이 종료되면 불완전한 파일이 생성될 수 있음

원인 분석

- 기존 파일을 덮어쓰는 도중 중단되면 파일이 손상될 수 있음

해결 방법

- 임시 파일(.tmp)에 기록을 완료한 뒤, File.Replace로 기존 저장 파일 교체

적용 효과

- **저장 중단으로 인한 기존 파일 손상 가능성 감소**

02 동시 저장 요청으로 인한 충돌 방지

저장 요청이 겹치면 임시 파일 쓰기가 충돌하거나
과거 데이터가 최신 데이터를 덮어쓸 수 있음

원인 분석

- 여러 저장 작업이 동일한 임시 파일(.tmp)에 동시 접근
- 비동기 저장의 완료 순서가 요청 순서와 달라질 수 있음

해결 방법

- 이전 저장 완료 후 다음 저장을 순차 실행하고, 실행 시점의 최신 데이터를 기록

적용 효과

- **동시 쓰기 충돌 방지 및 최신 저장 데이터 보호**